



권동우 | Frontend Developer

Email : hyeonggyugwon3@gmail.com

Phone : 010-8786-9260

GitHub : [@kwondongwoo0424](https://github.com/kwondongwoo0424)

Website : www.kwondongwoo.com

Blog : dlog.kwondongwoo.com

실험과 실패를 통해 성숙해지는 개발자

안녕하세요, 실패를 통해 더욱 성장할 수 있다고 믿는 **신입 개발자 권동우**입니다.

프론트엔드 기술에 흥미를 갖고 활발히 활동하고 있으며, 애자일과 MVP 방식으로 개발하는 것을 즐깁니다.

서비스 기획 / 디자인 / 개발 / 배포 / 운영 경험이 있습니다.

보다 나은 개발자가 되기 위해 **끊임없이 도전**하며, 실패 속에서 배움을 얻고 성장하고 있습니다.

프로젝트를 진행할때 **프로덕트 오너십**을 가지고 지속적으로 학습하고 UX를 개선합니다.

“나를 죽이지 못하는 것은 나를 더 강하게 만든다” 는 니체의 말처럼,

실패를 **성장의 발판**으로 삼아 더 나은 개발자로 나아가고 있습니다.

기술

언어 **JavaScript, TypeScript**, Java, Python

프론트엔드 **React**, Next.js, Vue.js, **styled-components**, tailwind CSS, Bootstrap, Tanstack Query, Zustand, Zod, Axios, Storybook, Electron

백엔드 Node.js, Express.js, Spring Boot

배포 및 도구 Git, Github, Figma, Postman, Docker, Supabase, Vercel, Netlify, Github Pages

인턴

XL8 2026.01.06. ~ 2026.02.27. (8주)

Webflow 기반 회사 랜딩페이지를 Supabase · Plasmic 환경으로 전환하는 마이그레이션 작업 참여

- Webflow CMS 데이터를 Supabase 데이터베이스 구조로 이전하고 데이터 관리 방식 개선
- Supabase DB 데이터를 Next.js에서 fetch하여 Plasmic에서 CMS 데이터로 활용할 수 있도록 데이터 연동 구조 구현
- 다국어 지원을 위한 Localization 구조 적용 및 반응형 디자인 구현

자격 및 역량 인증

구분	명칭	성과	일시
자격증	정보처리기사	2025년 4회 취득	2025.12.24.
역량 평가	TOPCIT 제23회	527점	2025.05.24.

프로젝트 목차

대표 프로젝트



Sync Desktop

비개발자와 개발자의 협업을 위한 AI 기반 프로젝트 관리 SaaS

팀 프로젝트
총원 7명

React

TypeScript

Electron

styled-components

TanStack Query

Zustand

데스크탑 앱 개발 / UI 디자인 / 디자인 시스템 구축 / 랜딩 페이지 개발 / SSE 기반 실시간 단어 해석 기능 개발



Dlog

개인 기술 블로그 사이트

개인 프로젝트
총원 1명

Vue.js

TypeScript

Bootstrap

Sanity CMS

TanStack Query

FSD 아키텍처 / 반응형 디자인 / 다크모드

사이드 프로젝트



DIA

대규모소프트웨어마이스터고 성적 가산출 서비스

팀 프로젝트
총원 4명

React

TypeScript

styled-components

React Context API

성적 계산 페이지 개발 / 성적 산출 알고리즘 모듈화



ColorVerse

색 조합을 추천해주는 웹 서비스

개인 프로젝트
총원 1명

React

styled-components

Express.js

OpenAI API

Express.js 서버 구축 / OpenAI API 통합 및 프롬프트 엔지니어링



Flick

학교 축제 부스 운영을 위한 전자 화폐형 실시간 거래·정산 플랫폼

팀 프로젝트
총원 5명

Next.js

Tailwind CSS

TypeScript

pnpm

어드민 페이지 실시간 부스 랭킹 시스템 개발



개인 웹 포트폴리오

개인 프로젝트
총원 1명

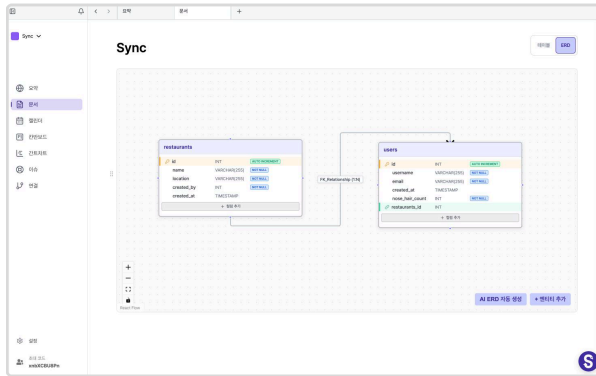
Next.js

Tailwind CSS

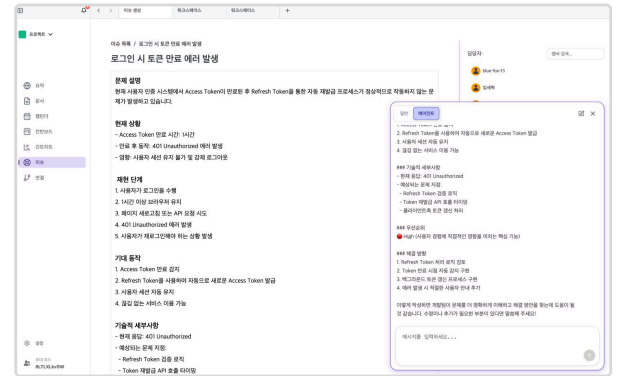
TypeScript

next-intl

한국어/영어 다국어 지원 / 검색 엔진 최적화 (SEO)



문서 페이지 - 기능명세서 기반 자동 ERD 생성



DeepSync - 이슈 내용 자동 수정

개요

디자이너, 기획자 등 비개발자와 개발자 간의 기술 격차로 인해 발생하는 의사소통 문제와 협업 비효율을 해결하기 위한 AI 기반 프로젝트 관리 데스크톱 앱입니다.
지식 격차를 AI가 보완하고, 팀원 간 실시간 협업 환경을 제공하는 것이 핵심 목표입니다.

핵심 문제

- 비개발자는 기술 지식 부족으로 요구사항 표현이 어려움
- 개발자는 비즈니스 맥락을 놓쳐 커뮤니케이션 비용 발생
- 협업 과정에서 잦은 오해, 시간 낭비, 반복 작업 발생

해결 아이디어

AI로 비개발자의 기술 이해도를 높이고,
실시간 협업 기능으로 개발자와의 커뮤니케이션을 최소 비용으로 만듭니다.

주요 기능

- 비개발자를 위한 드래그 기반 개발 단어 해석
- 프로젝트 컨텍스트를 이해하는 AI 챗봇으로 맞춤형 도움말 제공
- 실시간 협업이 가능한 마크다운 문서 페이지 및 기능명세서 기반 AI 자동 ERD 생성

팀원

- 총원 7명
프론트엔드 4명(본인포함), 백엔드 3명, 디자인 2명(본인포함)
본인 역할: **Frontend Developer & UI/UX Designer**

주요 사용 기술

React

상태 변화가 많은 화면을 빠르게 구성하기 위해 선택

TypeScript

협업과 유지보수에 필요한 타입 안정성을 확보하기 위해 사용

Electron

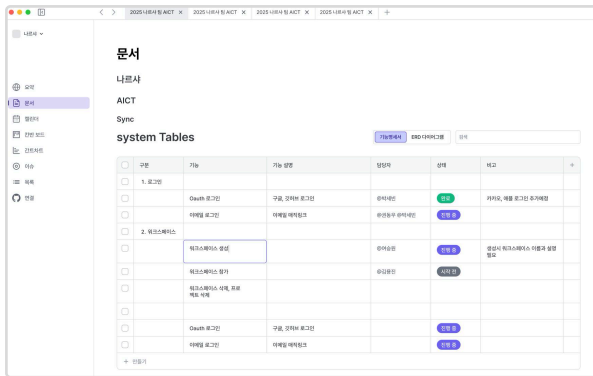
- 기존에 익숙한 웹 기술을 활용해 데스크탑 앱을 개발하기 위한
- macOS · Windows 등 다양한 환경에서 동일한 UX를 제공하기 위한 멀티 플랫폼 지원
- 브라우저 환경과 유사한 개발 흐름을 유지하며 개발 속도와 생산성을 확보할 수 있음

styled-components

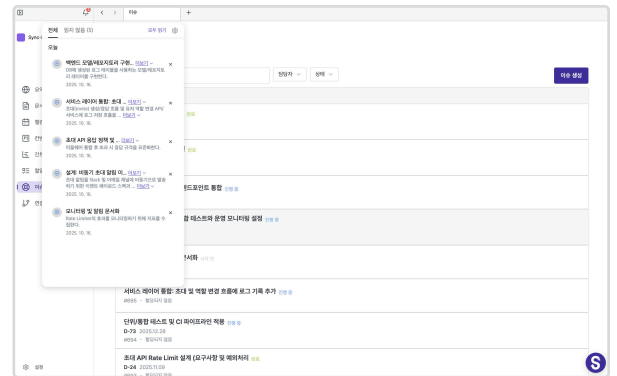
컴포넌트 단위로 스타일을 일관성 있게 관리하기 위해 사용

주요 활동 및 담당 역할

UI / UX 디자인



문서 페이지



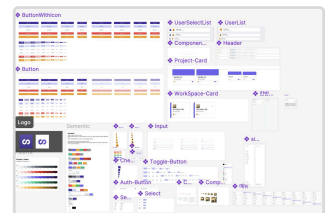
이슈 페이지

Figma의 Auto Layout 기능을 활용하여 사용자 경험을 고려한 화면을 설계

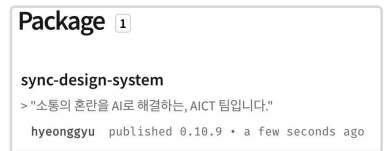
디자인 시스템 구축 (sync-design-system)

UI와 컴포넌트가 일관되지 않아 유지보수가 어렵고 재사용이 힘들었습니다.

- 재사용 가능한 15개 공통 컴포넌트 추출
 - **npm 패키지**(@sync-design-system)로 **배포**하여 팀 내 공유
 - **Storybook** 기반 **컴포넌트 문서화** 및 **사용 가이드 제공**
- 체계적인 컴포넌트 설계로 **개발 효율 향상** 및 **UI 일관성 확보**



디자인 시스템 구축 (Figma 활용)



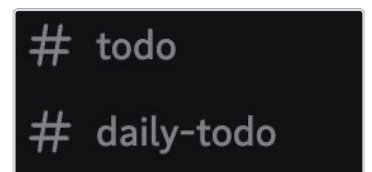
npmjs에 배포된 sds

애자일

애자일 개발론 적용

7개월 팀 프로젝트에서 초기 2개월간 명확한 마일스톤 없이 진행하다 주요 기능 지연 및 팀원 간 진행 상황 파악의 어려움이 발생했습니다.

- 1주 **스프린트 체계 도입**: 매주 목요일 데모 및 회고, 금요일 스프린트 계획
 - Notion로 백로그 관리, 스프린트 보드, 회고록 문서화
 - 디스코드로 데일리 리마인더, 블로킹 이슈 즉시 공유
- 장기 프로젝트에서 일정 예측 가능성과 팀 생산성을 높이는 경험 습득



daily-todo 작성 (디스코드 활용)



1주일 마다 스프린트 회고 작성

주요 활동 및 담당 역할

UX 개선

React Query 캐싱으로 이슈 페이지 로딩 시간 단축

사용자가 이슈 목록 ↔ 상세 페이지를 이동할 때마다 API 재호출
→ 평균 2초 로딩 소요, 같은 데이터를 반복 요청해 서버 부하 증가

- **React Query** 도입으로 이슈 **데이터** 자동 **캐싱**
- staleTime 2분 설정: 신선한 데이터는 재요청 없이 즉시 표시 (이슈 목록은 자주 변경되는 데이터이므로 staleTime를 짧게 설정함)
- 캐싱 전략으로 **성능**과 **사용자 경험**을 동시에 **개선**

낙관적 업데이트로 할 일 체크 반응성 개선

할 일 완료 체크 시 서버 응답 대기로 인한 0.5~1초 지연 발생
→ 사용자 경험 저하 발생

React Query **낙관적 업데이트 적용**으로 즉시 UI 반영

- **체감 응답속도** 평균 **800ms → 0ms**로 **단축**
- 네트워크 실패 시 자동 롤백 + 에러 토스트로 사용자 알림

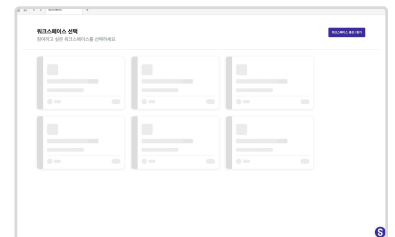


낙관적 업데이트 적용한 할 일 목록

불필요한 스켈레톤 UI 제거

워크스페이스 카드에 스켈레톤 UI 적용했으나
로딩 시 찰나에 나타났다 사라지는 플래시 현상 발생

- Chrome DevTools로 API 응답 시간 측정 결과 평균 180ms
- 스켈레톤 UI가 표시되자마자 사라져 시각적 노이즈로 작용
- 스켈레톤 UI 제거, 데이터 도착 즉시 콘텐츠를 렌더링하여
깜빡임 없이 **깔끔한 로딩 경험 제공**

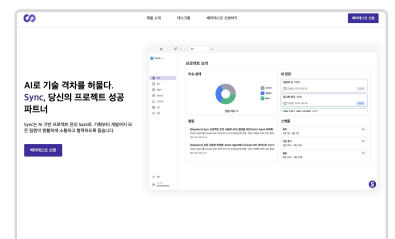


스켈레톤 UI가 적용된
워크스페이스 페이지

Sync introduce

베타테스트 랜딩 페이지(sync introduce) 개발

- GitHub Pages + 커스텀 도메인으로 베타테스트 신청 페이지 구축
- GitHub Releases 기반 배포로 Windows/macOS 설치 파일 제공
- 제품 검증을 위한 **효율적인 베타 배포 전략 수립** 및 실행 경험



Sync introduce

트러블 슈팅

SSE 스트리밍 응답의 과도한 리렌더링 최적화

문제 상황

SSE를 활용해 사용자가 텍스트를 드래그하면 실시간으로 AI가 단어를 설명해주는 기능을 구현하던 중, 응답이 화면에 표시되는 동안 화면이 끊기고 버벅거리는 문제 발생

원인

매 청크 수신마다 setState를 호출하여 과도한 리렌더링이 발생함

- n개 청크 수신 시 → n회 리렌더링 발생
- 각 렌더링마다 Virtual DOM 생성 및 Diffing 과정이 수행됨

→ 성능 저하

```
onChunk: (chunk) => {  
  // 현재 처리 중인 단어가 맞는지 확인  
  if (currentWordRef.current !== word) {  
    return; // 다른 단어의 응답이면 무시  
  }  
  
  // 매 청크마다 즉시 상태 업데이트  
  setDefinition(prev => prev + chunk);  
},
```

기존 코드

해결 과정

이 문제를 해결하기 위해 Throttling과 Debouncing의 두 가지 방식을 고민

1. Throttling

- 일정 간격마다 렌더링을 시도하기 때문에 타이핑 효과가 일정하고 부드러움
- 그러나 새로운 청크가 도착하지 않는 구간에서도 주기적으로 렌더링이 발생하여, 불필요한 렌더링이 발생됨

2. Debouncing

- 청크가 연속적으로 수신되는 동안에는 렌더링을 지연
- 일정 시간동안 새로운 청크가 수신되지 않을 때 한 번만 렌더링 실행

Throttling은 타이핑 애니메이션이 자연스럽다는 장점이 있었지만, 불필요한 렌더링이 발생해 성능적인 문제가 남아 있었음.

반면 Debouncing은 타이핑 효과는 약간 덜 자연스러울 수 있으나, 렌더링 횟수를 크게 줄여 UI 끊김 문제를 가장 효과적으로 해소할 수 있었음.

이러한 이유로 최종적으로 Debouncing 방식을 선택

해결 방법

Debouncing + useRef 기반 버퍼링 패턴 적용

1. useRef를 중간 버퍼로 활용 → 리렌더링 없이 데이터 누적
2. 50ms Debouncing 적용 → 연속 청크 도착 시 렌더링 지연
3. 이전 타이머 취소 → 마지막 청크 기준으로 한 번만 setState 호출

→ 청크는 실시간으로 버퍼에 쌓이지만, 50ms 동안 추가 청크가 없을 때만 화면 갱신되도록 최적화

```
onChunk: (chunk) => {  
  // 현재 처리 중인 단어가 맞는지 확인  
  if (currentWordRef.current !== word) {  
    return; // 다른 단어의 응답이면 무시  
  }  
  
  definitionRef.current += chunk;  
  
  if (updateTimerRef.current) {  
    clearTimeout(updateTimerRef.current);  
  }  
  
  updateTimerRef.current = setTimeout(() => {  
    // 다시 한번 현재 단어 확인  
    if (currentWordRef.current === word) {  
      setDefinition(definitionRef.current);  
    }  
  }, 50);  
},
```

Debouncing 방식을 적용한 코드

결과 및 배운점

- setState 호출 횟수 대폭 감소로 UI 끊김 문제 해결
- Throttling, Debouncing 같은 렌더링 최적화 기법 실전 적용 경험
- React 렌더링 메커니즘에 대한 깊이 있는 이해

느낀점 & 참고 자료

느낀점

7개월 동안 팀 프로젝트를 진행하며 팀워크는 저절로 만들어지지 않는다는 것을 깨달았습니다. 서로의 경험이 달라 의견 충돌도 있었고, 의도를 충분히 이해하지 못해 어려움을 겪기도 했습니다.

하지만 대화를 통해 작은 오해도 금방 풀 수 있었고, 문제 해결보다 중요한 것은 서로를 이해하려는 태도라는 것을 배웠습니다.

이번 프로젝트를 통해 기능을 만드는 것뿐 아니라, 팀이 함께 성장하는 과정의 가치를 느낄 수 있었습니다.

성과

FIX 2025 대한민국 ICT 융합 엑스포 부스 운영

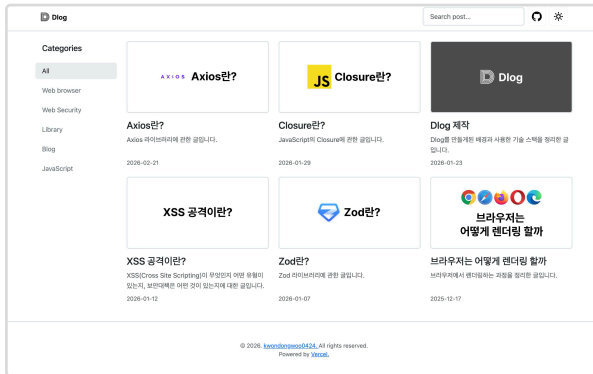
2025.10.22.~2025.10.25.

2025 ITCE 프로젝트 3등 수상

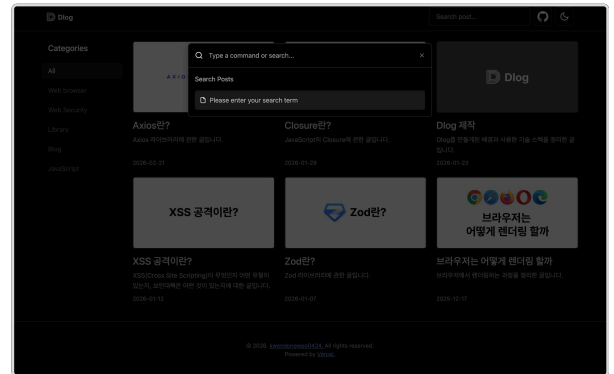
2025.10.24.

참고 자료

Sync-Desktop 소스코드	: github.com/AICT-SYNC/sync-desktop
Sync Design System 소스코드	: github.com/AICT-SYNC/sync-design-system
Sync Design System NPM	: npmjs.com/package/sync-design-system
Sync Introduce 소스코드	: github.com/AICT-SYNC/sync-introduce
sync-saas 홈페이지(서비스 종료)	: https://sync-saas.com
팀 포트폴리오	: https://buly.kr/FWUeFtb



홈 화면 - 라이트 모드



홈 검색 화면 - 다크 모드

개요

기존에는 TIL(Today I Learned)를 작성하고 GitHub에 커밋하여 글을 관리했습니다. 하지만 파일 관리 번거로움과 카테고리/검색 기능 부재라는 한계를 해결하고 Vue.js와 Bootstrap을 학습하기 위해 Sanity CMS 기반 개인 블로그를 개발했습니다. 백엔드 없이 콘텐츠를 관리할 수 있으며, 현재 학습 내용 정리에 실사용 중입니다.

팀원

- 총원 1명 (개인 프로젝트)

주요 사용 기술

Vue.js

TypeScript

Bootstrap

Sanity CMS

TanStack Query

느낀 점

단순히 기능을 구현하는 것을 넘어, 아키텍처 설계부터 SEO, 실서비스 운영까지 직접 경험하며 프론트엔드 개발의 전체 흐름을 이해할 수 있었습니다. 특히 FSD 아키텍처를 적용하면서 코드 구조에 대해 고민하는 습관이 생겼고, 실제 사용자 유입 데이터를 보며 개발이 끝이 아닌 운영까지 이어진다는 것을 체감했습니다.

참고 자료

Dlog 웹 페이지 : dlog.kwondongwoo.com
Dlog-web 소스코드 : github.com/kwondongwoo0424/Dlog
Dlog-sanity 소스코드 : github.com/kwondongwoo0424/Dlog-sanity

주요 활동 및 담당 역할

아키텍처 Feature-Sliced Design 아키텍처 적용

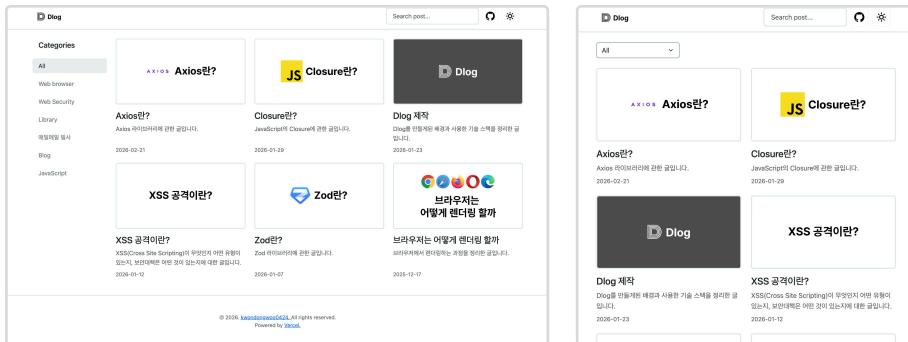
컴포넌트 간 의존성이 불명확하고 기능이 추가될수록 코드 위치를 찾기 어려운 문제가 있었습니다.

- pages / widgets / entities / shared 레이어로 역할 분리
 - 상위 레이어만 하위 레이어를 참조하는 단방향 의존성 규칙 적용
 - 기능별 응집도 향상으로 코드 추가 및 수정 위치를 직관적으로 파악 가능
- 일관된 구조로 **유지보수성**과 **확장성 확보**

UI/UX 반응형 레이아웃으로 디바이스별 최적화된 UX 제공

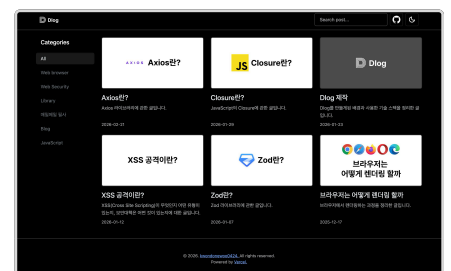
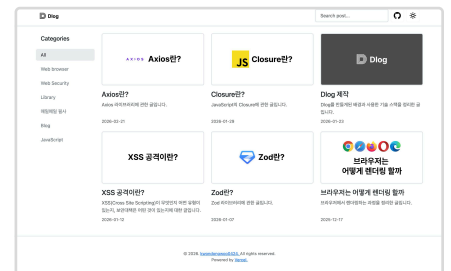
데스크탑과 모바일에서 동일한 UI를 사용하면 화면 크기에 따라 사용성이 떨어지는 문제가 있었습니다.

- 데스크탑: 사이드바에 카테고리 목록을 항상 노출해 빠른 탐색 가능
 - 모바일: 사이드바 대신 카테고리 드롭다운으로 전환해 화면 공간 효율 확보
 - CSS 미디어 쿼리 기반으로 breakpoint에 따라 컴포넌트 분기 처리
- **디바이스 특성에 맞는 UI**로 UX 개선



다크모드 구현 - 시스템 설정 감지 및 사용자 설정 저장

- prefers-color-scheme 미디어 쿼리로 시스템 다크모드 설정 자동 감지
 - localStorage에 사용자 선택 저장, 재방문 시 이전 설정 유지
 - Bootstrap data-bs-theme 속성 기반으로 전체 테마 일괄 전환
- 첫 방문 시 시스템 설정에 맞게 자동 적용, 이후 사용자 설정 우선 유지



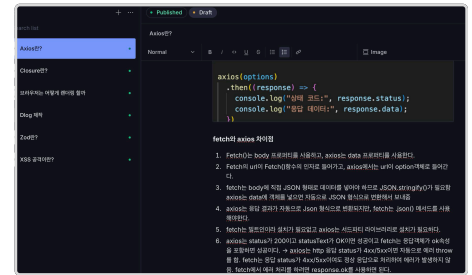
주요 활동 및 담당 역할

콘텐츠 관리 Headless CMS(Sanity) 콘텐츠 관리 환경 구축

백엔드 서버 구축·운영에 드는 개발 시간과 비용을 최소화하면서 콘텐츠를 관리할 수 있는 구조가 필요했습니다.

- Sanity Headless CMS 도입으로 별도 서버 없이 콘텐츠 관리 환경 구축
- Sanity Studio에서 글 작성·수정·삭제 시 재배포 없이 즉시 반영
- GROQ 쿼리로 카테고리 필터링, 최신순 정렬 등 유연한 데이터 조회 구현

→ 서버 개발·배포·유지보수 없이 웹 개발에 집중, 서버 운영 비용 절감



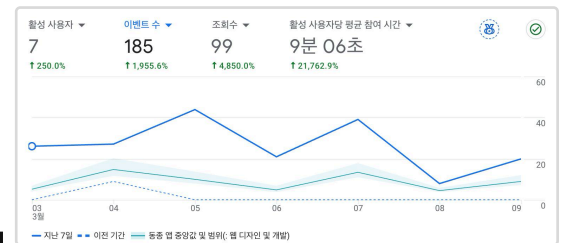
Sanity Studio 글 작성 화면

운영 성과 실제 서비스 운영 및 사용자 유입 확인

단순 개발로 끝내지 않고 실제 서비스로 운영하며 사용자 유입을 확인했습니다.

- 현재 블로그 포스트 7개 작성 및 발행
- Google Analytics를 통해 사용자 유입 데이터 분석

→ 직접 콘텐츠를 작성·운영하며 **실서비스 경험 추적**



Google Analytics 대시보드

트러블 슈팅

검색엔진 포스트 페이지 미색인 문제

문제 상황

웹 사이트 배포 후 Google Search Console 및 site:dlog.kwondongwoo.com 검색을 통해 모니터링한 결과, 홈페이지(/)만 색인이 생성되고 포스트 상세 페이지(/post/{slug})들은 색인이 전혀 생성되지 않는 것을 확인했습니다.

원인

public/sitemap.xml을 정적으로 작성하면서 홈 URL (/)만 포함되어 있었습니다. Googlebot은 sitemap을 참고해 크롤링 대상을 탐색하기 때문에, 포스트 페이지들이 sitemap에 등록되지 않아 검색 엔진이 해당 페이지들을 인식하지 못하는 문제가 발생했습니다.

해결 과정

sitemap에 포스트 URL을 포함시키는 방법으로 아래 두 가지를 비교하였습니다.

방법 1 - 빌드 타임 sitemap 자동 생성

- 배포 시점에 Sanity에서 포스트 목록을 조회해 sitemap 파일을 자동으로 생성하는 방식
- 빌드 이후에 새 글을 작성하면 다음 배포 전까지 sitemap에 포함되지 않는 문제가 있음
- 글 발행 시마다 재배포를 트리거해야 하는데, 콘텐츠 업데이트 때마다 불필요한 배포가 반복되는 구조가 됨

방법 2 - 서버리스 함수로 동적 sitemap 생성

- 요청이 들어올 때마다 Sanity에서 실시간으로 포스트/카테고리를 조회해 XML을 생성하는 방식
- 재배포 없이 새 글이 즉시 sitemap에 반영됨

Sanity CMS에서 콘텐츠를 관리하는 구조상 글 작성이 배포와 분리되어 있기 때문에, 빌드 타임 방식을 선택하면 새 글을 올릴 때마다 재배포가 필요해 CMS를 도입한 이점이 사라진다고 판단하여 방법 2를 채택하였습니다.

해결 방법

Vercel 서버리스 함수(api/sitemap.xml.ts)를 작성하여 /sitemap.xml 요청을 처리하도록 구성

Sanity에서 전체 포스트(slug, publishedAt, _updatedAt)와 카테고리(slug) 데이터를 조회한 뒤, 홈, 카테고리, 포스트 페이지를 포함한 Sitemap XML을 동적으로 생성하여 반환

기존 public/sitemap.xml 정적 파일이 Vercel에서 서버리스 함수보다 우선순위가 높아 덮어씌워지는 문제가 있어 정적 파일을 삭제하고 vercel.json에 rewrites 설정을 추가해 /sitemap.xml 요청이 명시적으로 서버리스 함수로 라우팅되도록 처리

결과 및 배운점

- Google Search Console에 sitemap 등록 후 포스트 페이지가 순차적으로 인덱싱되기 시작했고, 이후 새 글 작성 시 재배포 없이도 다음 크롤링에서 자동으로 sitemap에 반영되도록 개선
- sitemap을 단순 정적 파일로 생성하는 방식보다 콘텐츠 업데이트 방식에 맞는 전략 선택이 중요함을 확인
- 정적 sitemap과 동적 sitemap 방식의 트레이드오프를 비교하며 SEO 관점과 인프라 구조를 함께 고려하는 설계의 필요성을 학습

사이드 프로젝트



DIA

대구소프트웨어마이스터고 성적 가산출 서비스

2025.08.09. ~ 2025.09.09.

주요 업무

- 성적 계산 페이지
- 성적 산출 알고리즘 모듈화

주요 기술

React

TypeScript

styled-components

참고 자료

- 프론트 소스 코드
: github.com/EntryCNS/DIA
- 웹 페이지
: <https://trydgs.com>

DIA 졸업 예정자 성적 입력 페이지

개요

대구소프트웨어마이스터고 1차 입학 전형의 성적 반영 방식을 기반으로, 학생이 교과 성적을 입력하면 총점을 산출해주는 간단하고 직관적인 웹 서비스입니다.

팀원

- 총원 4명
- 웹 프론트엔드 4명(본인포함)

배운점

React Context API를 사용해 전역 상태 관리

학생 유형 선택 → 성적 입력 → 결과 확인까지 3단계 폼 개발 중

- 5~6개 컴포넌트를 거쳐 props 전달 (5단계 depth)
- 새 데이터 필드 추가 시 6개 파일 모두 수정 필요

→ **React Context API**로 전역 상태 관리 중앙화

- 실시간 동기화: 여러 컴포넌트에서 동일한 상태 공유
- **Props Drilling 제거**: 5단계 전달 → 직접 접근으로 개선

→ 체계적인 상태 관리 설계 및 유지보수 용이한 아키텍처 구축

Redux, Zustand 대신 React Context API 선택 이유

- 프로젝트 요구사항이 "여러 페이지 간 폼 데이터 공유" 정도로 단순하여, Redux의 action/reducer 구조나 Zustand의 selector 기반 구조까지 필요하지 않았음
- 상태 변경 빈도가 낮아 Redux 또는 Zustand의 미세한 상태 분리 기능이 필요하지 않았고, Context API만으로도 충분한 성능을 확보할 수 있었음

```
Before (Props Drilling)
[App] - grades, setGrades, freeSem, attendance... (12개 props)
↓
[Router] - (12개 props 그대로 전달)
↓
[StudentWritePage] - (12개 props 그대로 전달)
↓
[Body] props만 전달, 사용 안함 - (12개 props 그대로 전달)
↓
[WriteGrade] - grades, setGrades 사용
↓
[WriteGradeListItem] - grades 사용
```



```
After (Context API)
[App]
└─ [ScoreProvider] 전역 상태
    ├── [StudentWritePage] → useScore()
    ├── [WriteGrade] → useScore()
    └── [WriteGradeListItem] → useScore()
```




주요 기술

React

styled-components

Express.js

OpenAI API

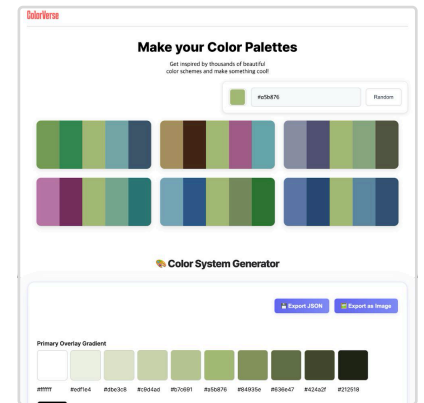
참고 자료

- 소스 코드

: github.com/kwondongwoo0424/ColorVerse

- 노션 문서

: <https://buly.kr/GvoBKj0>



홈 화면

개요

디자이너와 개발자가 일관된 색상 시스템을 빠르게 구축하고,
AI의 도움을 받아 조화로운 색상 조합을 찾을 수 있도록 도와주는 웹 서비스입니다.

팀원

- 총원 1명 (개인 프로젝트)

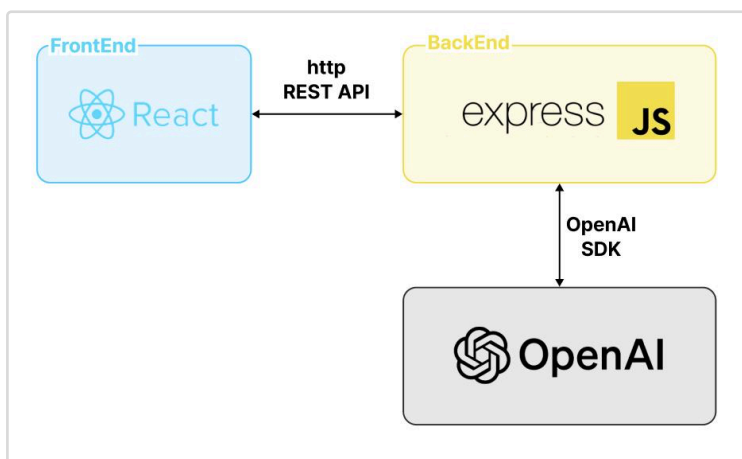
배운점

Express.js를 활용한 RESTful API 서버 구축

- /api/chat POST 엔드포인트로 OpenAI API 연동
- CORS 설정으로 프론트엔드 - 백엔드 간 통신 구현

OpenAI API 통합 및 프롬프트 엔지니어링

- OpenAI 공식 Node.js SDK(openai)로 GPT-3.5-turbo 모델 연동
- API 키 인증 → 요청 구성 → 응답 처리까지 전체 호출 흐름 구현



사이드 프로젝트



Flick

학교 축제 부스 운영을 위한 전자 화폐형 실시간 거래·정산 플랫폼

2025.05.05. ~ 2025.05.20.

주요 업무

- 어드민 페이지 개발

주요 기술

Next.js

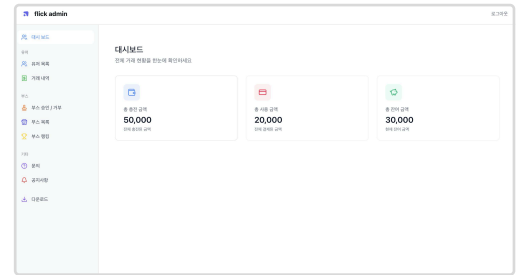
tailwind CSS

TypeScript

참고 자료

- 프론트 소스 코드

: github.com/EntryCNS/flick-web



Flick 어드민 대시보드 페이지

개요

Flick은 교내 축제 부스의 결제를 위한 포인트 기반 거래 서비스입니다.

이전에 선배들이 개발한 v1은 QR 코드 결제 방식에서 몇 가지 불편함이 있었고, 이를 개선하고 사용자 경험을 향상시키기 위해 Flick v2를 새롭게 개발하였습니다.

팀원

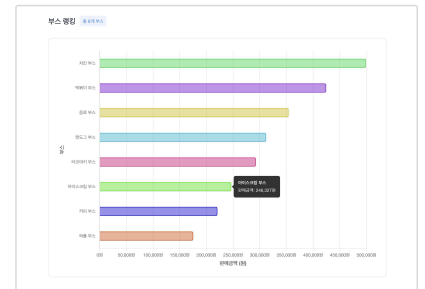
- 총원 5명

웹 프론트엔드 2명(본인포함), 백엔드 2명, 풀스택 1명

웹 개발

실시간 부스 랭킹 시스템 개발

- Chart.js와 **WebSocket**을 활용한 실시간 매출 순위 시각화 시스템 구축
- WebSocket 재연결 로직으로 안정적인 실시간 데이터 통신 구현



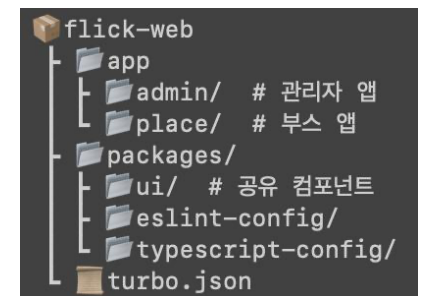
어드민 랭킹 페이지

배운점

프로젝트의 코드베이스 통합 및 관리 효율화

Turborepo와 pnpm Workspace 기반의 **모노레포 아키텍처**로 관리자 대시보드와 부스 운영자용 두 개의 Next.js 앱을 통합 관리하며, 공통 코드를 재사용해 개발 효율을 높였습니다.

- @repo/ui: 공통 컴포넌트 중앙화로 코드 중복 제거
 - pnpm 하드링크로 node_modules 용량 절감
 - workspace:* 프로토콜로 패키지 간 의존성 자동 관리
- 멀티 프로젝트 환경에서 생산성과 일관성을 동시에 확보한 경험



flick-web 프로젝트 구조



주요 기술

Next.js

tailwind CSS

TypeScript

next-intl

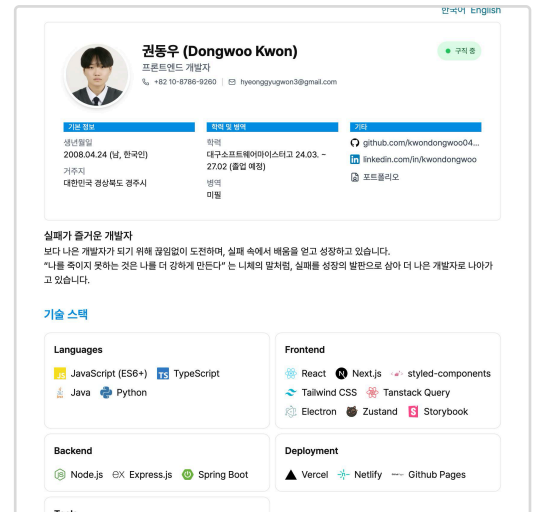
참고 자료

- 프론트 소스 코드
: github.com/kwondongwoo0424/portfolio

- 웹 사이트 주소
: <https://kwondongwoo.com>

개요

기존에 사용하던 노션 포트폴리오의
디자인 자유도 부족과 SEO 미지원 등의 한계를 느끼고,
Next.js App Router 기반으로 개인 포트폴리오
웹사이트를 직접 제작했습니다.



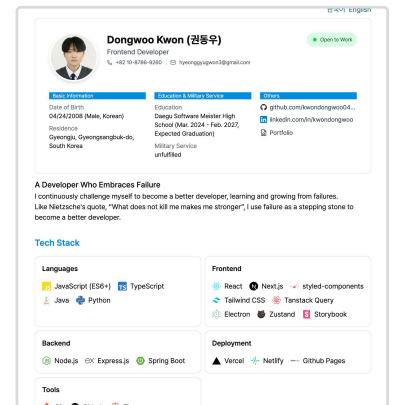
홈 화면

웹 개발

i18n 한국어/영어 다국어 지원

App Router 및 Server Component 호환성을 고려해
next-intl 채택

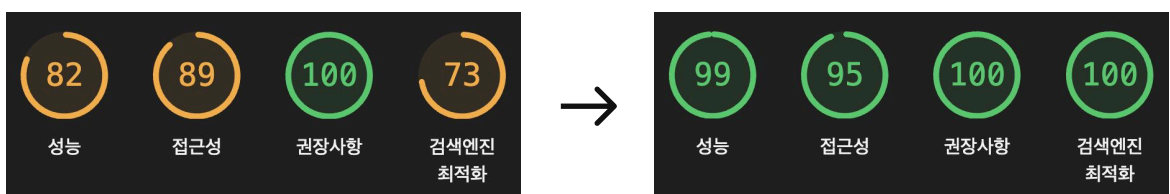
- [locale] 동적 세그먼트를 활용해 /ko, /en URL 기반으로 언어별 라우팅 처리
- Vercel + next-intl 조합으로 Accept-Language 헤더 기반 자동 언어 리다이렉트 구현
- 언어 전환 버튼은 usePathname()으로 현재 경로를 감지해 URL만 교체하는 방식으로 구현, 페이지 재로드 없이 전환 가능



홈 화면 /en 경로

SEO 최적화

- SSR을 지원하는 Next.js를 채택해 검색엔진 크롤러가 JavaScript 실행 없이도 메타태그와 콘텐츠를 수집할 수 있도록 구성
- generateMetadata()를 활용해 로케일별(/ko, /en) 메타태그를 분리 생성
- hreflang alternates를 적용해 검색엔진이 언어별 페이지를 정확히 인식하도록 처리 (x-default 포함)
- sitemap.ts, robots.ts API로 sitemap.xml과 robots.txt를 동적 생성



Light-House 측정 결과

학력

대구소프트웨어마이스터고등학교

2024.03. ~ 2027.02. (졸업예정)

파이썬 프로그래밍 / 웹 프로그래밍 기초 / 데이터베이스 프로그래밍 / 자료구조 / 인공지능 기초 / 소프트웨어 공학 / 자바 프로그래밍 / 웹 프로그래밍 / 네트워크 / 웹 프로그래밍 실무 / 자바 프로그래밍 실무

수상

2025 교내 교과 우수상 (한국사)	2025.12.22.
2025 ITCE 프로젝트 3등 수상	2025.10.24.
2025 교내 교과 우수상 (운동)	2025.7.22.

활동

실리콘밸리 온라인 인턴십 - XL8	2026.01.06. ~ 2026.02.27.
2025학년도 SW마이스터고 연합 해커톤 참가	2025.11.05. ~ 2025.11.07.
FIX 2025 대한민국 ICT 융합 엑스포 부스 운영	2025.10.22. ~ 2025.10.25.
2025 교내 소프트웨어 해커톤 참가	2025.07.15. ~ 2025.07.16.
2025 교내 나르샤 프로젝트 참가 - Sync Desktop	2025.03.17. ~ 2025.10.25.
2025 교내 글로벌 1:1 원어민 화상영어 참가	2025.05.13. ~ 2025.12.23.
교내 도서부 부장	2025.03. ~ 2025.12.
2025학년도 신입생 2차 면접 봉사 참여	2024.10.25.
2024 교내 나르샤 프로젝트 참가	2024.08.20. ~ 2024.12.26.
2024 교내 소프트웨어 해커톤 참가	2024.07.17. ~ 2024.07.18.
2024 교내 글로벌 1:1 원어민 화상영어 참가	2024.05.8. ~ 2025.01.08.
교내 전공 동아리 연합 코딩테스트 참가	2024.04.17.
2024 스마틴앱 챌린지 참가	2024.04.02.
교내 전공 동아리 CNS 입부	2024.03.14.

끝까지 읽어주셔서 감사합니다.

아직 많이 부족하지만, 매일 한 걸음씩 성장하는 개발자 권동우가 되겠습니다.